

CPSC 416: Design Project 3 - Primary Backup/Replication

Background

Context: We are tasked with creating a fault-tolerant key value store via primary/backup server replication. This service will maintain consistency across its primary and backup nodes, and ensure resilience to network partitions - in-turn sacrificing availability. We follow requirements specified as part of the lab instructions. We will discuss the general ideas, assumptions, failure models, invariants, protocol sketch, and observability plan of this design.

General idea: At its core, our implementation involves a “master” ViewServer which is responsible for managing the system, by coordinating the primary and backup key/value servers which perform core system functionality. The major components of our system are the client, servers, and a distinct singular ViewServer. It is assumed that the client will respect the rules of our system, as in they will not act in a “byzantine” fashion (maliciously finding hacks, or bypassing security measures), and interact with the servers via standard public APIs for managing key-value stores. Our key-value store will be an append-only store, this is to ensure ease of consistency. Thus, we allow the general APIs of get, put, update but not delete. The servers are split up into 3 distinct categories; primary, backup, and idle. Primary servers are the first-responders, they are the contact point between our system and the clients and are responsible for communicating downstream to accomplish replication as well as returning the value to the client. Upon receiving a request, before updating the application state, the primary server will forward the request to the backup server. The backup server does not do any real processing that is relevant to the client, it will simply, in-order, save the operation in its log, then apply the update to its application state and return a suitable response (success or failure) to the primary. The primary will then update its own log, and finally its application state, before returning the response to the client (if everything has been successful). Idle servers are those servers which are currently “alive,” as in available and ready to work, but have no assigned role. The ViewServer is a special master server responsible for assigning and keeping track of Primary/Backup server role assignment. Upon failure, the ViewServer is the one responsible for recovering the system via its backup assignment to primary.

Failure Model: We focus on the “timing failure” model. This is because in our system, it is impossible to distinguish between servers that have crashed or servers that are temporarily unavailable. We are not guaranteed a reliable system, our messages can be reordered, lost, or delayed, we are unsure if they will, or when they will reach the desired endpoint and when we will get back a response. Thus, we bound our system on response timing. Specifically, servers will send heartbeat pings to the ViewServer, ViewServer responds with the most updated view and also keeps track of “alive” servers. We will also bind response time between other protocols, for example the client and our servers, or the primary and backup, ensuring we don’t get stuck on any one request at any point in our system - as long as the system is recoverable at that point given our assumptions/invariants.

Assumptions:

- ViewServer is not replicated, and therefore a potential point of failure, we assume it is reliable.
- Primary and backup servers operate in a single-threaded environment, that is one-at a time. Furthermore, there is only ever one primary or backup server active at any one point in time.
- We must copy the entirety of the application state from the primary, even if the backup is almost up to date.
- Data is stored in volatile memory, that is if both primary and backup servers crash we cannot recover.
- ViewServer cannot make progress if primary fails before acknowledging the view in which its primary.
- If primary fails with no backup, we will not go back to uninitialized state (test spec).

Desired Properties from Doc:

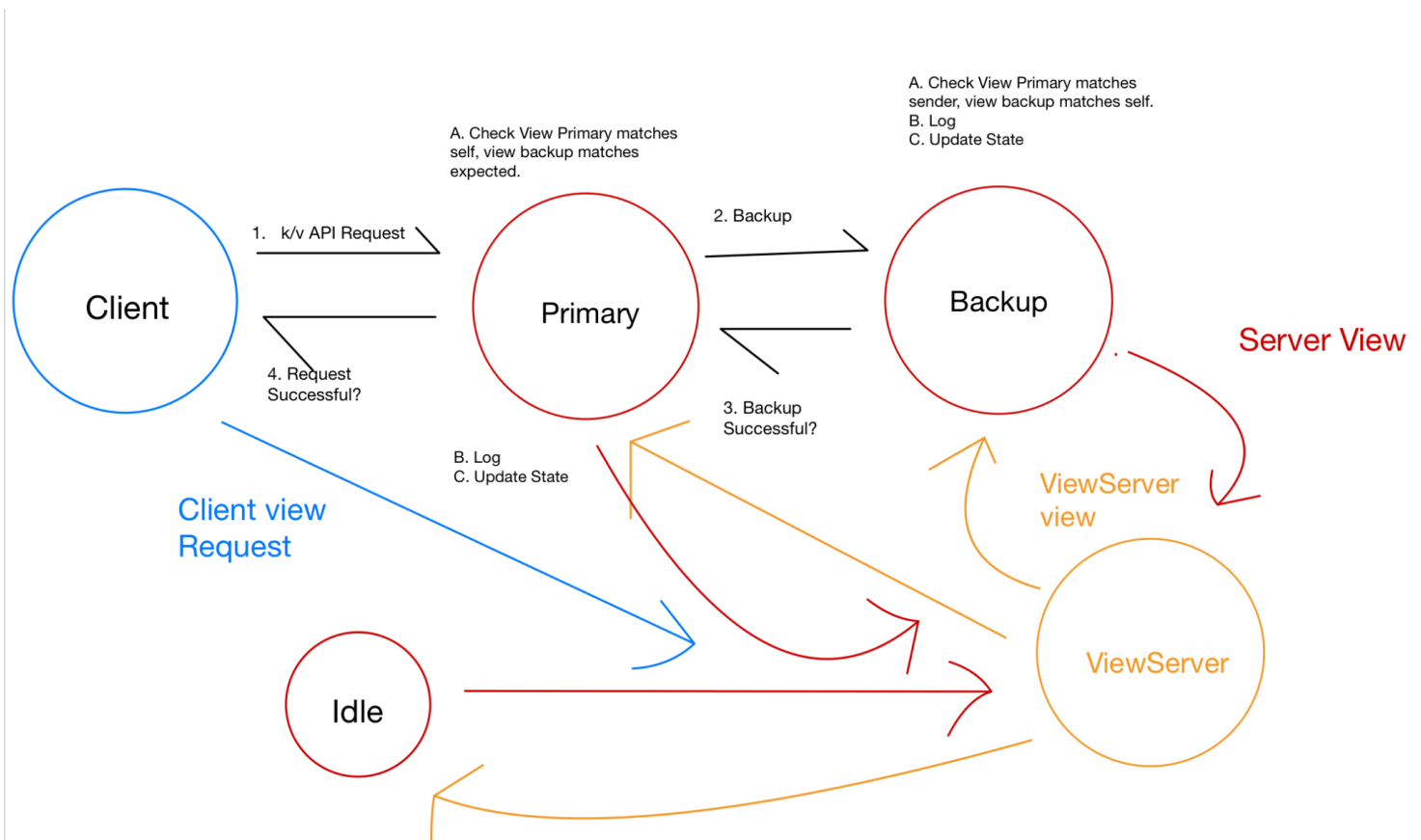
- The ViewServer cannot change views until it receives an acknowledgement for the current view from the primary of current view.
- The primary in a view must always be the primary or backup from the previous view. We ensure only one server change between two consecutive views.
- Backup can be any “alive” server other than primary, or none if none is available.
- Message and time handlers should be deterministic.
- Service should be consistent.
- Service should be resilient to network partitions.
- Operations should execute linearizably, that is one-at-a-time we can order them sequentially and reconstruct the state.
- Operations should provide exactly-once semantics, that is repeated instructions (perhaps from a duplicate request) do not cause changes in the system.

Deduced Invariants:

- Single-primary commit: only a server in the primary role may complete a client operation.
- No split brain completion: only one server can complete an operation, and that operation must be in the same position/order across all logs.
- Backup Sync Completion: When a new backup is installed, we do not count as backup until sync completes.
- Exactly-once effect: each client request changes the store at most once.

Liveness is achieved if our assumptions are met, that is we have a reliable ViewServer and at least one active server. Eventually, via the retries, we will complete requests.

Protocol Design



The above diagram describes the rough successful case of our design. It proceeds as follows:

1. Client makes an API request to the primary.
2. Primary ensures the view specifies itself as primary, if so we proceed further and request backup. If backup exists:
 - a. The backup does the same: it ensures the view primary matches the sender and itself is backed up.
 - b. Backup logs
 - c. Backup update its own state
 - d. Backup sends response (success or failure) to primary.
3. Primary interprets response, depending on response it will log and then update.
4. Primary sends the client its response.

In the background all k/v servers are pinging the ViewServer every PING_MILLIS. ViewServer responds to those servers with ViewReply. Additionally, every single request (ie. client->primary, primary->backup) even if not explicitly mentioned, if expecting a response has a timeout which will induce exponential retry upon it being reached.

The above sketch is a high-level abstraction of what will be going on behind the scenes. It does not illustrate how we will deal with unique cases, those being initialization and recovery from failure.

Initialization:

- All servers ping the ViewServer, with ViewServer.STARTUP_VIEWNUM. The server makes the first server it processes with this response as the primary. Returning to all servers a tuple of the form {ViewNum, Primary, Backup}. The first response specifically will be {1, SelectedPrimary, Null}. The ViewServer will hang until that SelectedPrimary acknowledges itself as primary by returning to the ViewServer the same view in its ping. Once that has happened, the system can be considered initialized. If that does not happen, we have entered one of our assumed failure states we cannot recover from.
- All incoming clients initially call GetView on the ViewServer, ideally just once, to grab the primary and backup server address to direct requests to.
- If other servers are available, after the first view has been acknowledged with the primary, the ViewServer can select a backup. Update its view, and send to its servers as {2, SelectedPrimary, SelectedBackup}.
 - We routinely do this any time the backup is null. We must stop the world here, syncing between primary and backup before updating the tuple and allowing new requests.

Failure Cases Communication:

- Client:
 - Fails to receive any response from the server it contacts.
 - Solution: Client retries the request with exponential backoff. After exceeding 3 iterations, it requests an updated view from ViewServer via GetView(). It resets its interval upon receiving the response, and continues to cycle.
 - Part 2: If we exceed X seconds without a differing response, the network may be partitioned. We should attempt the backup.
 - Receives Failure Response:
 - Operation Rejected:
 - Solution: No issue here, this can happen if the API request was invalid.
 - Wrong Server Error:

- This will be caused by the client having a stale address that it's treating as primary. It will call GetView() to get an updated primary address and repeat its request.
- Busy:
 - Depending on the decided complexity, it could be the case that the server is currently busy responding to another request.
 - Solution: We have to decide whether to buffer the requests on the server side or reject outright. I suggest buffering client requests, we will still respond busy when we reach a set limit. Thus the client should retry.
- Internal Error:
 - This response will be received if there is some inconsistency in the system that prevents the server from executing the request, this is not the clients responsibility. But they will retry, we can consider forcing a sync via the ViewServer if this continues to re-establish a consistent state through the log.
- Primary:
 - Current View does not indicate itself as primary:
 - Solution: The primary sends back an operation rejected. It requests an updated view from ViewServer.
 - Fails to receive any response from backup:
 - Solution: Primary retries the request with exponential backup. After exceeding 3 iterations, it requests an updated view from ViewServer via GetView(). If the received backup is not Null it keeps retrying. If Null, it avoids the backup pathway logging and updating locally only before returning a response.
 - Note, backup timer should be longer than the PING_MILLIS to avoid exhausting resources.
 - Request Rejected:
 - Primary does not match:
 - This is caused by the backup node having a different view than the primary server. The backup node will reject the request.
 - Solution: We can request a new view from the ViewServer() if it matches us, we can request backup again.
 - Corrupted Message:
 - We will use a next sequence number generator, this should match for the operation between Primary and Backup to ensure replication correctness. Potentially we can include this in the view, or as part of the request between primary and backup. The mechanism we use is operation sequence number, this will also help with linearizability. We must ensure the requested operation will have the same operation sequence number as the desired in the primary.
 - Solution: cause manual sync
 - Busy:
- Backup:
 - Receives client request:
 - The backup should never receive client requests, the only time it may potentially encounter this is if the network is partitioned and the primary is unreachable.
 - Solution: The backup should indicate to ViewServer that the network is partitioned and it needs to select a new primary. Again, we assume the ViewServer is always reachable/reliable.

Other Failure Cases:

- Primary passes two consecutive intervals without pinging ViewServer, we assume primary has crashed:
 - A backup exists:
 - Solution: The ViewServer updates the tuple {ViewNum, Primary, Backup} to {ViewNum + 1, Backup (as new primary), Null}. Setting the prior backup to the primary now.
 - Note: we may lose information, as we do not know if the primary received new operations.
 - No backup exists:
 - The system has entered an irrecoverable failure state as stated in the invariants.
- Backup Failure:
 - An idle server exists:
 - Solution: ViewServer promote idle server to new backup, sync, and update views.
 - Note: sync request timeout should be larger than normal to avoid exhausting resources
- Network is partitioned:
 - The primary may be unreachable, in which case the client can attempt to request the backup.
 - Solution: Although the backup rejects the request, it will inform the ViewServer which can select to update the primary to an available node.
- No available nodes:
 - We can't do anything
- Primary isn't acknowledging new view:
 - We can't do anything

Observability Plan

We will be logging server response metrics, this includes the types of responses each server is sending (ie. response successful, vs rejected or etc.) as well as the average latency to respond to different types of client requests. That way if there is some issue slowing down our system we can quickly observe it. Some examples of logs we may find useful:

view changes received: (node, oldView $\rightarrow$ newView, role change)

client ops: (clientId, reqId, op, viewUsed)

forwarded ops: (seq, clientId, reqId)

sync start/finish: (view, snapshotId/hash, lastSeq)

rejects: (reason=InvalidView | NotPrimary | NotBackup, localView, senderView)

Other Ideas

- Can we compress the logs at all at some point? For example, by writing to disk and having the servers use that to startup so they can start hot. Additionally, allows us to reduce the amount we have to sync each time as the log can grow easily.
- Singleton design for generating operation sequence number and view number.
- Idempotency keys can be utilized to prevent operation retries twice, our current append only implementation will already avoid the only once problem but this can ensure extreme observance of this protocol. This will require overhead in the servers to know if its responded to a request.